



PET BAKIM YÖNETİM SİSTEMİ (VETCARE)

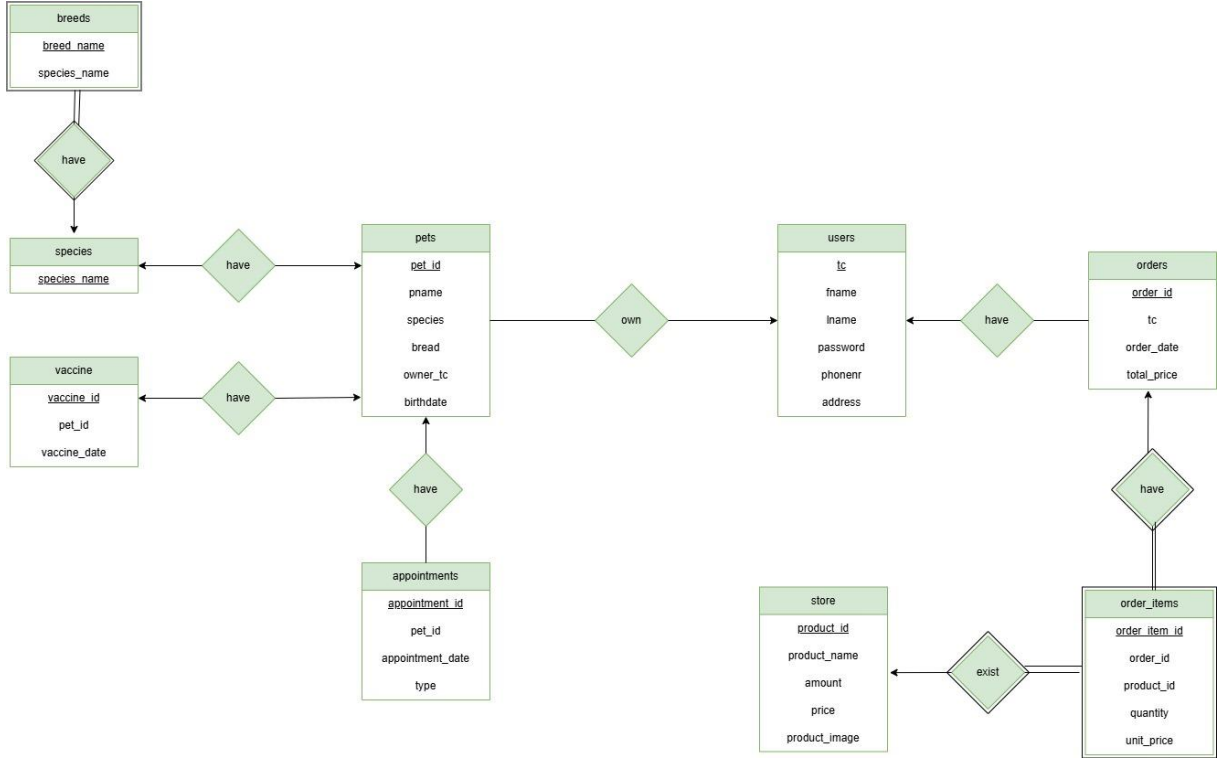
Öğrenciler:

Alperen Kayım – 21011048

Türkan Sena Yücel – 21011078

Berat Can Karakaş – 21011102

ER DİYAGRAMI:



1. Oluşturacağınız veritabanı en az 4 tablo içermelidir. Her tabloda en az 10 kayıt bulunmalıdır.

Appointments tablosu:

	appointment_id [PK] integer	pet_id integer	appointment_date timestamp without time zone	type character varying (10)
1	1	1	2023-02-01 14:30:00	Muayene
2	2	2	2023-02-10 15:00:00	Aşı
3	3	3	2023-03-01 10:00:00	Kontrol
4	4	4	2023-03-15 11:30:00	Muayene
5	5	5	2023-04-01 09:00:00	Aşı
6	6	6	2023-04-20 16:00:00	Kontrol
7	7	7	2023-05-01 10:30:00	Muayene
8	8	8	2023-05-15 11:00:00	Aşı
9	9	9	2023-06-01 14:00:00	Kontrol
10	10	10	2023-06-20 15:30:00	Muayene
11	11	11	2023-07-01 09:30:00	Aşı
12	12	12	2023-07-15 16:30:00	Kontrol
13	13	13	2023-08-01 10:15:00	Muayene
14	14	14	2023-08-20 14:45:00	Aşı
15	15	15	2023-09-01 13:00:00	Kontrol

Breeds tablosu:

	breed_name [PK] character varying (20) 	species_name character varying (20) 
1	Amazon Papağanı	Papağan
2	Ankara Kedisi	Kedi
3	Ardenne	At
4	Chihuahua	Köpek
5	Dalmaçyalı	Köpek
6	Fırat Yılanı	Yılan
7	Golden Retriever	Köpek
8	Habeş Kedisi	Kedi
9	Holstein	İnek
10	Japon Balığı	Balık
11	Labrador	Köpek
12	Muhabbet Kuşu	Kuş
13	Sivas Kangalı	Köpek
14	Siyah Tetra	Balık
15	Van Kedisi	Kedi

Order_items tablosu:

	order_item_id [PK] integer 	order_id integer 	product_id integer 	quantity integer 	unit_price numeric (10,2) 
1	1	1	1	2	50.00
2	2	2	2	1	70.00
3	3	3	3	5	30.00
4	4	4	4	1	25.00
5	5	5	5	3	40.00
6	6	6	6	2	35.00
7	7	7	7	1	120.00
8	8	8	8	1	150.00
9	9	9	9	2	60.00
10	10	10	10	10	10.00
11	11	11	11	3	20.00
12	12	12	12	2	15.00
13	13	13	13	1	50.00
14	14	14	14	1	55.00
15	15	15	15	1	200.00

Orders tablosu:

	order_id [PK] integer	tc character varying (11)	order_date timestamp without time zone	total_price numeric (10,2)
1	1	12345678901	2023-01-01 00:00:00	100.00
2	2	12345678902	2023-01-05 00:00:00	150.00
3	3	12345678903	2023-01-10 00:00:00	80.00
4	4	12345678904	2023-01-15 00:00:00	200.00
5	5	12345678905	2023-01-20 00:00:00	120.00
6	6	12345678906	2023-01-25 00:00:00	90.00
7	7	12345678907	2023-01-30 00:00:00	75.00
8	8	12345678908	2023-02-01 00:00:00	110.00
9	9	12345678909	2023-02-05 00:00:00	140.00
10	10	12345678910	2023-02-10 00:00:00	100.00
11	11	12345678911	2023-02-15 00:00:00	180.00
12	12	12345678912	2023-02-20 00:00:00	160.00
13	13	12345678913	2023-02-25 00:00:00	200.00
14	14	12345678914	2023-03-01 00:00:00	90.00
15	15	12345678915	2023-03-05 00:00:00	50.00

Pets tablosu:

	pet_id [PK] integer	pname character varying (20)	species character varying (20)	breed character varying (20)	owner_tc character (11)	birthdate date
1	1	Maviş	Kedi	Van Kedisi	12345678901	2020-03-15
2	2	Karabaş	Köpek	Sivas Kangalı	12345678902	2019-07-20
3	3	Boncuk	Kuş	Muhabbet Kuşu	12345678903	2021-05-10
4	4	Balık	Balık	Japon Baliği	12345678904	2022-06-25
5	5	Leo	Köpek	Golden Retriever	12345678905	2018-09-12
6	6	Pamuk	Kedi	Ankara Kedisi	12345678906	2020-11-30
7	7	Fıstık	Köpek	Chihuahua	12345678907	2021-01-10
8	8	Renkli	Papağan	Amazon Papağanı	12345678908	2019-03-08
9	9	Holstein1	İnek	Holstein	12345678909	2015-12-05
10	10	Şimşek	At	Ardenner	12345678910	2013-08-01
11	11	Neon	Balık	Siyah Tetra	12345678911	2022-10-21
12	12	Kıvrım	Yılan	Fırat Yılanı	12345678912	2017-02-11
13	13	Ares	Kedi	Habeş Kedisi	12345678913	2018-05-19
14	14	Benek	Köpek	Dalmaçyalı	12345678914	2021-07-14
15	15	Max	Köpek	Labrador	12345678915	2016-09-22

Species tablosu:

	species_name [PK] character varying (20) 
1	At
2	Balık
3	Deve
4	Fok
5	Hamster
6	İnek
7	Kaplumbağa
8	Kedi
9	Kertenkele
10	Köpek
11	Kuş
12	Maymun
13	Papağan
14	Tavşan
15	Yılan

Store tablosu:

	product_id [PK] integer 	product_name character varying (30) 	amount integer 	price numeric (10,2) 	product_image bytea 
1	1	Kedi Maması	100	50.00	[null]
2	2	Köpek Maması	80	70.00	[null]
3	3	Balık Yemi	150	30.00	[null]
4	4	Tasma	50	25.00	[null]
5	5	Kedi Kum Kabı	30	40.00	[null]
6	6	Köpek Oyuncağı	60	35.00	[null]
7	7	Kuş Kafesi	20	120.00	[null]
8	8	Kedi Tırmalama Tahtası	15	150.00	[null]
9	9	Papağan Yemi	100	60.00	[null]
10	10	Kemik	200	10.00	[null]
11	11	Su Kabı	70	20.00	[null]
12	12	Tarak	40	15.00	[null]
13	13	Kedi Şampuanı	30	50.00	[null]
14	14	Köpek Şampuanı	19	55.00	[null]
15	15	Yılan Kafesi	8	200.00	[null]

Users tablosu:

	tc [PK] character (11) 	fname character varying (20) 	lname character varying (20) 	password character varying (20) 	phonelnr character varying (12) 	address text 
1	12345678901	Ali	Kaya	password1	5551234567	Ankara
2	12345678902	Ayşe	Yılmaz	password2	5551234568	İstanbul
3	12345678903	Mehmet	Demir	password3	5551234569	İzmir
4	12345678904	Fatma	Çelik	password4	5551234570	Bursa
5	12345678905	Ahmet	Şahin	password5	5551234571	Adana
6	12345678906	Emine	Öztürk	password6	5551234572	Antalya
7	12345678907	Hasan	Erdoğan	password7	5551234573	Kayseri
8	12345678908	Hülya	Kara	password8	5551234574	Trabzon
9	12345678909	Cem	Güneş	password9	5551234575	Konya
10	12345678910	Zeynep	Taş	password10	5551234576	Mersin
11	12345678911	Burak	Aydın	password11	5551234577	Gaziantep
12	12345678912	Elif	Ekin	password12	5551234578	Samsun
13	12345678913	Murat	Ak	password13	5551234579	Eskişehir
14	12345678914	Derya	Bayrak	password14	5551234580	Çanakkale
15	12345678915	Serkan	Yıldız	password15	5551234581	Edirne

Vaccine tablosu:

	vaccine_id [PK] integer 	pet_id integer 	vaccine_date timestamp without time zone 
1	1	1	[null]
2	2	2	[null]
3	3	3	[null]
4	4	4	[null]
5	5	5	[null]
6	6	6	[null]
7	7	7	[null]
8	8	8	[null]
9	9	9	[null]
10	10	10	[null]
11	11	11	[null]
12	12	12	[null]
13	13	13	[null]
14	14	14	[null]
15	15	15	[null]

2. Tablolarınızda primary key ve foreign key kısıtlarını kullanmalısınız.

```
CREATE TABLE users (
    tc CHAR(11) PRIMARY KEY,          -- TC kimlik numarası, 11 haneli
    fname VARCHAR(20) NOT NULL,       -- Ad, 50 karaktere kadar
    lname VARCHAR(20) NOT NULL,       -- Soyad, 50 karaktere kadar
    password VARCHAR(20) NOT NULL,    -- Şifre, 100 karaktere kadar
    phonenr VARCHAR(12) NOT NULL,     -- Telefon numarası, 15 karaktere kadar
    address TEXT NOT NULL             -- Adres, metin türü
);

CREATE TABLE species (
    species_name VARCHAR(20) PRIMARY KEY -- animal_name yalnızca bu sütun ve birincil anahtar olarak tanımlandı
);

CREATE TABLE breeds (
    breed_name VARCHAR(20) PRIMARY KEY, -- Birincil anahtar
    species_name VARCHAR(20) NOT NULL,   -- Yabancı anahtar
    FOREIGN KEY (species_name) REFERENCES species(species_name) ON DELETE CASCADE -- species tablosuna referans
);

CREATE TABLE pets (
    pet_id SERIAL PRIMARY KEY,          -- Pet ID, otomatik artan birincil anahtar
    pname VARCHAR(20) NOT NULL,         -- Pet adı, 20 karaktere kadar
    species VARCHAR(20) NOT NULL,        -- Tür adı, 20 karaktere kadar
    breed VARCHAR(20) NOT NULL,          -- Irk adı, 20 karaktere kadar
    owner_tc CHAR(11) NOT NULL,          -- Sahibinin TC kimlik numarası, 11 haneli
    birthdate DATE NOT NULL,            -- Doğum tarihi, tarih formatında
    FOREIGN KEY (owner_tc) REFERENCES users(tc) ON DELETE RESTRICT, -- RESTRICT kısıtlaması eklendi
    FOREIGN KEY (species) REFERENCES species(species_name) ON DELETE CASCADE, -- Tür adı animals tablosuna referans
    FOREIGN KEY (breed) REFERENCES breeds(breed_name) ON DELETE CASCADE -- Tür adı animals tablosuna referans
);

CREATE TABLE vaccine (
    vaccine_id SERIAL PRIMARY KEY,      -- Benzersiz aşı kimliği
    pet_id INT NOT NULL,               -- Aşı yapılan hayvanın kimliği (foreign key ile ilişkilendirilebilir)
    vaccine_date TIMESTAMP,            -- Aşının yapıldığı tarih ve saat
    FOREIGN KEY (pet_id) REFERENCES pets(pet_id) ON DELETE CASCADE -- Hayvan silinirse ilişkili kayıtlar da silinir
);

CREATE TABLE appointments (
    appointment_id SERIAL PRIMARY KEY, -- appointment_id, otomatik artan bir birincil anahtar
    pet_id INT NOT NULL,               -- pet_id, ilgili pet'in ID'si
    appointment_date TIMESTAMP NOT NULL, -- appointment_date, randevu tarihi ve saati
    type VARCHAR(10) NOT NULL,         -- type, randevu türü (örneğin: muayene, aşı vb.)
    FOREIGN KEY (pet_id) REFERENCES pets(pet_id) ON DELETE CASCADE -- pet_id, pets tablosuna referans verir
);

CREATE TABLE store (
    product_id SERIAL PRIMARY KEY,      -- Benzersiz ürün kimliği
    product_name VARCHAR(30) NOT NULL,  -- Ürün adı
    amount INT NOT NULL,               -- Ürün miktarı (stokta bulunan)
    price NUMERIC(10, 2) NOT NULL,      -- Ürünün fiyatı
    product_image BYTEA                -- Ürün resmi (genellikle bir URL veya dosya yolu)
);

CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,        -- order_id, otomatik artan bir birincil anahtar
    tc VARCHAR(11) NOT NULL,           -- tc, kullanıcı TC kimlik numarası
    order_date TIMESTAMP NOT NULL,     -- order_date, siparişin tarihi ve saati
    total_price NUMERIC(10, 2) NOT NULL, -- total_price, siparişin toplam fiyatı (10 basamağa kadar, 2 basamağı ondalıklı)
    FOREIGN KEY (tc) REFERENCES users(tc) ON DELETE CASCADE -- tc, users tablosuna referans verir
);

CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,   -- order_item_id, her bir sipariş öğesine benzersiz bir kimlik atar ve otomatik artar
    order_id INT NOT NULL,              -- order_id, siparişin kimliği (foreign key)
    product_id INT NOT NULL,            -- product_id, ürünün kimliği (foreign key)
    quantity INT NOT NULL,             -- quantity, sipariş edilen ürün miktarı
    unit_price NUMERIC(10, 2) NOT NULL, -- unit_price, ürünün birim fiyatı
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE, -- order_id, orders tablosuna referans verir
    FOREIGN KEY (product_id) REFERENCES store(product_id) ON DELETE RESTRICT -- product_id, store tablosuna referans verir
);
```

3. En az bir tabloda silme kısıtı ve sayı kısıtı olmalıdır.

orders tablosunda hem silme kısıtı hem de toplam fiyatın negatif olup olmadığını kontrol eden bir sayı kısıtı vardır.

```
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,          -- order_id, otomatik artan bir birincil anahtar  
    tc VARCHAR(11) NOT NULL,              -- tc, kullanıcı TC kimlik numarası  
    order_date TIMESTAMP NOT NULL,        -- order_date, siparişin tarihi ve saati  
    total_price NUMERIC(10, 2) NOT NULL,   -- total_price, siparişin toplam fiyatı (10 basamağa kadar, 2 basamağı ondalıklı)  
    FOREIGN KEY (tc) REFERENCES users(tc) ON DELETE CASCADE, -- tc, users tablosuna referans verir  
    CONSTRAINT chk_total_price_positive CHECK (total_price >= 0) -- Toplam fiyat negatif olamaz  
);
```

4. Arayüzden en az birer tane INSERT, UPDATE ve DELETE işlemi gerçekleştirilebilmelidir.

```
String insertQuery = "INSERT INTO store (product_name, amount, price, product_image) VALUES (?, ?, ?, ?)";
```

```
String deleteQuery = "DELETE FROM store WHERE product_id = ?";
```

```
String query = "UPDATE store SET amount = amount - ? WHERE product_id = ?";
```


5. Arayüzden girilecek bir değere göre ekrana sonuçların listelendiği bir sorgu yazmalısınız.

Admin panelinde kullanıcıları isimlerine göre aramaya yarayan işlem.

```
CREATE TYPE user_d AS (  
    tc VARCHAR(20),  
    fname VARCHAR(20),  
    lname VARCHAR(20),  
    phonenr VARCHAR(20),  
    address TEXT,  
    pet_count INTEGER  
);  
  
CREATE OR REPLACE FUNCTION search_users_array(search_text VARCHAR)  
RETURNS user_d[] AS  
$$  
DECLARE  
    cur_users CURSOR FOR  
        SELECT  
            u.tc,  
            u.fname,  
            u.lname,  
            u.phonenr,  
            u.address,  
            (SELECT COUNT(*) FROM pets p WHERE p.owner_tc = u.tc) AS pet_count  
        FROM users u  
        WHERE LOWER(tc) LIKE LOWER('%' || search_text || '%' ) OR  
              LOWER(fname) LIKE LOWER('%' || search_text || '%' ) OR  
              LOWER(lname) LIKE LOWER('%' || search_text || '%' ) OR  
              LOWER(phonenr) LIKE LOWER('%' || search_text || '%' ) OR  
              LOWER(address) LIKE LOWER('%' || search_text || '%' );  
  
    result_array user_d[] := '{}';  
    i INTEGER := 1;  
BEGIN  
    FOR rec IN cur_users LOOP  
        result_array := array_append(result_array, NULL::user_d);  
  
        result_array[i].tc := rec.tc;  
        result_array[i].fname := rec.fname;  
        result_array[i].lname := rec.lname;  
        result_array[i].phonenr := rec.phonenr;  
        result_array[i].address := rec.address;  
        result_array[i].pet_count := rec.pet_count;  
  
        i := i + 1;  
    END LOOP;  
  
    RETURN result_array;  
END;  
$$ LANGUAGE plpgsql;
```

```
private void searchUsers(String searchText, DataTableModel model) {  
    model.setRowCount(0); // Newcut satırları temizle  
    String query = "SELECT * FROM unnest(search_users_array(?)) AS (tc VARCHAR(20), fname VARCHAR(20), lname VARCHAR(20), phonenr VARCHAR(20), address TEXT, pet_count INTEGER)";  
  
    try (PreparedStatement stmt = Database.getConnection().prepareStatement(query)) {  
        stmt.setString(1, searchText); // Arama metnini parametre olarak ekle  
  
        ResultSet rs = stmt.executeQuery();  
        while (rs.next()) {  
            String tc = rs.getString("tc");  
            String fname = rs.getString("fname");  
            String lname = rs.getString("lname");  
            String phone = rs.getString("phonenr");  
            String address = rs.getString("address");  
            int petCount = rs.getInt("pet_count");  
  
            // Sonuçlara tabloya ekle  
            model.addRow(new Object[]{tc, fname, lname, phone, address, petCount});  
        }  
    } catch (SQLException ex) {  
        ex.printStackTrace();  
        JOptionPane.showMessageDialog(  
            null,  
            "Database Error: " + ex.getMessage(),  
            "Error",  
            JOptionPane.ERROR_MESSAGE  
        );  
    }  
}
```

User Management System

<

USER

TC	NAME	SURNAME	PHONENR	ADDRESS	NBR OF PETS
12345678907	Hasan	Erdoğan	5551234573	Kayseri	1

SEARCH

Hasan ismi aratılınca çıkan kullanıcılar

6. Arayüzden çağrılan sorgulardan en az biri "VIEW" olarak tanımlanmış olmalıdır.

```
CREATE OR REPLACE FUNCTION create_user_pets_view(current_tc VARCHAR(11))
RETURNS VOID AS $$
BEGIN
    EXECUTE format(
        'CREATE OR REPLACE VIEW user_pets_view AS
        SELECT * FROM pets WHERE owner_tc = %L',
        current_tc
    );
END;
$$ LANGUAGE plpgsql;
```

```
String petsViewQuery = "SELECT create_user_pets_view('" + currentUserTc + "')";
```

User Management System

<

Your Pets

ID	PET NAME	SPECIES	BREED	BIRTHDATE
1	Maviş	Kedi	Van Kedisi	2020-03-15
47	karabaj	Köpek	Sivas Kangalı	2018-10-09

ADD

7. En az bir adet "SEQUENCE" oluřturmalı ve arayüzden yapılacak INSERT sırasında ilgili sütundaki değerlerin otomatik olarak atanmasını saęlamalısınız.

```
CREATE SEQUENCE pet_id_seq START 16;

CREATE TABLE pets (
  pet_id INT PRIMARY KEY DEFAULT NEXTVAL('pet_id_seq'), -- Sequence kullanılarak pet_id otomatik artacak
  pname VARCHAR(20) NOT NULL, -- Pet adı, 20 karaktere kadar
  species VARCHAR(20) NOT NULL, -- Tür adı, 20 karaktere kadar
  breed VARCHAR(20) NOT NULL, -- Irk adı, 20 karaktere kadar
  owner_tc CHAR(11) NOT NULL, -- Sahibinin TC kimlik numarası, 11 haneli
  birthdate DATE NOT NULL, -- Doğum tarihi, tarih formatında
  FOREIGN KEY (owner_tc) REFERENCES users(tc) ON DELETE RESTRICT, -- RESTRICT kısıtlaması eklendi
  FOREIGN KEY (species) REFERENCES species(species_name) ON DELETE CASCADE, -- Tür adı species tablosuna referans
  FOREIGN KEY (breed) REFERENCES breeds(breed_name) ON DELETE CASCADE -- Irk adı breeds tablosuna referans
);
```

User Management System

<

Your Pets

ID	PET NAME	SPECIES	BREED	BIRTHDATE
1	Maviř	Kedi	Van Kedisi	2020-03-15
47	karabaj	Köpek	Sivas Kangalı	2018-10-09

ADD

8. Arayüzden çağrılan sorgulardan en az birinde UNION veya INTERSECT veya EXCEPT kullanmış olmalısınız.

```
String vaccineViewQuery = "CREATE OR REPLACE VIEW user_vaccines_view AS " +  
    "SELECT p.pname, p.species, p.breed, v.vaccine_date " +  
    "FROM vaccine v " +  
    "JOIN pets p ON v.pet_id = p.pet_id " +  
    "WHERE v.vaccine_date IS NULL AND p.owner_tc = '" + currentUserTc + "'" +  
    "UNION " +  
    "SELECT p.pname, p.species, p.breed, v.vaccine_date " +  
    "FROM vaccine v " +  
    "JOIN pets p ON v.pet_id = p.pet_id " +  
    "WHERE v.vaccine_date <= NOW() - INTERVAL '2 months' AND p.owner_tc = '" + currentUserTc + "'";
```

9. Sorgularınızın en az biri aggregate fonksiyonlar içermeli, HAVING ifadesi kullanılmalıdır.

Hayvanı olmayan kullanıcılar user listesinde admin tarafından aranmadıkça gözükmez.

```
String query = ""  
    SELECT  
        u.tc,  
        u.fname,  
        u.lname,  
        u.phonenr,  
        u.address,  
        COUNT(p.owner_tc) AS pet_count  
    FROM users u  
    LEFT JOIN pets p ON u.tc = p.owner_tc  
    GROUP BY u.tc, u.fname, u.lname, u.phonenr, u.address  
    HAVING COUNT(p.owner_tc) > 0  
    "";
```

TC	NAME	SURNAME	PHONENR	ADDRESS	NR OF PETS
12345678909	Cem	Güneş	5551234575	Konya	1
12345678908	Hülya	Kara	5551234574	Trabzon	1
12345678910	Zeynep	Taş	5551234576	Mersin	1
12345678904	Fatma	Çelik	5551234570	Bursa	1
12345678906	Emine	Öztürk	5551234572	Antalya	1
12345678914	Derya	Bayrak	5551234580	Çanakkale	1
12345678911	Burak	Aydın	5551234577	Gaziantep	1
12345678915	Serkan	Yıldız	5551234581	Edirne	1
12345678912	Elif	Ekin	5551234578	Samsun	1
12345678905	Ahmet	Şahin	5551234571	Adana	1
12345678903	Mehmet	Demir	5551234569	İzmir	1
12345678907	Hasan	Erdoğan	5551234573	Kayseri	1
12345678902	Ayşe	Yılmaz	5551234568	İstanbul	1
12345678913	Murat	Ak	5551234579	Eskişehir	1
12345678901	Ali	Kaya	5551234567	Ankara	2

10. Arayüzden girilen değerleri parametre olarak alıp ekrana sonuç döndüren 3 farklı SQL fonksiyonu tanımlamış olmalısınız. Bu fonksiyonlarda en az birer tane "RECORD" ve "CURSOR" tanımı-kullanımı olmalıdır.

```
private void searchPets(String searchText, DefaultTableModel model) {
    model.setRowCount(0); // Mevcut satırları temizle
    String query = "SELECT * FROM search_pets_array(?) AS (pet_id INTEGER, pname VARCHAR(20), owner_tc VARCHAR(20), species VARCHAR(20), breed VARCHAR(20), birthDate DATE)";

    try (PreparedStatement stmt = Database.getConnection().prepareStatement(query)) {
        stmt.setString(1, searchText); // Arama metnini parametre olarak ekle

        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            int id = rs.getInt("pet_id");
            String ownerTc = rs.getString("owner_tc");
            String name = rs.getString("pname");
            String species = rs.getString("species");
            String breed = rs.getString("breed");
            Date birthDate = rs.getDate("birthDate");

            // Sonuçları tabloya ekle
            model.addRow(new Object[]{id, ownerTc, name, species, breed, birthDate});
        }
    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(
            null,
            "Error searching pets: " + ex.getMessage(),
            "Error",
            JOptionPane.ERROR_MESSAGE
        );
    }
}
```

```
CREATE TYPE pets_d AS (
    pet_id INTEGER,
    pname VARCHAR(20),
    species VARCHAR(20),
    breed VARCHAR(20),
    birthdate DATE
);

CREATE OR REPLACE FUNCTION search_pets_array(search_text VARCHAR)
RETURNS pet_d[] AS
$$
DECLARE
    cur_pets CURSOR FOR
        SELECT pet_id, pname, species, breed, birthdate
        FROM pets
        WHERE LOWER(CAST(pet_id AS CHAR)) LIKE LOWER('%' || search_text || '%' ) OR
              LOWER(pname) LIKE LOWER('%' || search_text || '%' ) OR
              LOWER(species) LIKE LOWER('%' || search_text || '%' ) OR
              LOWER(breed) LIKE LOWER('%' || search_text || '%' ) OR
              LOWER(CAST(birthdate AS CHAR)) LIKE LOWER('%' || search_text || '%' );

    result_array pet_d[] := '{}';
    i INTEGER := 1;
BEGIN
    FOR rec IN cur_pets LOOP
        result_array := array_append(result_array, NULL::pet_d);

        result_array[i].pet_id := rec.pet_id;
        result_array[i].pname := rec.pname;
        result_array[i].species := rec.species;
        result_array[i].breed := rec.breed;
        result_array[i].birthdate := rec.birthdate;

        i := i + 1;
    END LOOP;

    RETURN result_array;
END;
$$ LANGUAGE plpgsql;
```

ID	OWNER TC	PET NAME	SPECIES	BREED	BIRTHDATE
15	12345678915	Max	Köpek	Labrador	2016-09-22

Max

SEARCH

Max diye aratılınca çıkan sonuç

```
private void searchOrders(String searchText, DefaultTableModel model) {
    model.setRowCount(0); // Mevcut satırları temizle
    String query = "SELECT * FROM unnest(search_orders_array(?)) AS (order_id INTEGER, tc VARCHAR(20), order_date DATE, total_price NUMERIC)";

    try (PreparedStatement stmt = Database.getConnection().prepareStatement(query)) {
        stmt.setString(1, searchText); // Arama metnini parametre olarak ekle

        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            int orderId = rs.getInt("order_id");
            String tc = rs.getString("tc");
            Date orderDate = rs.getDate("order_date");
            double totalPrice = rs.getDouble("total_price");

            // Sonuçları tabloya ekle
            model.addRow(new Object[]{orderId, tc, orderDate, totalPrice});
        }
    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(
            null,
            "Error searching orders: " + ex.getMessage(),
            "Error",
            JOptionPane.ERROR_MESSAGE
        );
    }
}

}
```

```
CREATE TYPE order_d AS (
    order_id INTEGER,
    tc VARCHAR(20),
    order_date DATE,
    total_price NUMERIC
);

CREATE OR REPLACE FUNCTION search_orders_array(search_text VARCHAR)
RETURNS order_d[] AS
$$
DECLARE
    cur_orders CURSOR FOR
        SELECT order_id, tc, order_date, total_price
        FROM orders
        WHERE LOWER(CAST(order_id AS CHAR)) LIKE LOWER('%' || search_text || '%') OR
              LOWER(tc) LIKE LOWER('%' || search_text || '%') OR
              LOWER(CAST(order_date AS CHAR)) LIKE LOWER('%' || search_text || '%') OR
              LOWER(CAST(total_price AS CHAR)) LIKE LOWER('%' || search_text || '%');

    result_array order_d[] := '{}';
    i INTEGER := 1;
BEGIN
    FOR rec IN cur_orders LOOP

        result_array := array_append(result_array, NULL::order_d);

        result_array[i].order_id := rec.order_id;
        result_array[i].tc := rec.tc;
        result_array[i].order_date := rec.order_date;
        result_array[i].total_price := rec.total_price;

        i := i + 1;
    END LOOP;

    RETURN result_array;
END;
$$ LANGUAGE plpgsql;
```

ORDERS			
TC	ID	TOTAL \$	DATE
1	12345678901	2023-01-01	100.0
17	12345678901	2025-01-06	300.0
18	12345678901	2025-01-06	70.0

Belli bir tc aratılınca çıkan orderlar

```
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    model.setRowCount(0); // Mevcut satırları temizle
    String query = "SELECT * FROM unnest(search_users_array(?)) AS (tc VARCHAR(20), fname VARCHAR(20), lname VARCHAR(20), phoneN VARCHAR(20), address TEXT, pet_count INTEGER)";

    try {
        PreparedStatement stat = Database.getConnection().prepareStatement(query);
        stat.setString(1, searchText); // Arama metnini parametre olarak ekle

        ResultSet rs = stat.executeQuery();
        while (rs.next()) {
            String tc = rs.getString("tc");
            String fname = rs.getString("fname");
            String lname = rs.getString("lname");
            String phone = rs.getString("phoneN");
            String address = rs.getString("address");
            int petCount = rs.getInt("pet_count");

            // Sonuçları tabloya ekle
            model.addRow(new Object[] {tc, fname, lname, phone, address, petCount});
        }
    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(
            null,
            "Database Error: " + ex.getMessage(),
            "Error",
            JOptionPane.ERROR_MESSAGE
        );
    }
}
```

User Management System

USER

TC	NAME	SURNAME	PHONE NR.	ADDRESS	NR OF PETS
12345678907	Hasan	Erdoğan	5551234573	Kayseri	1

SEARCH

Hasan ismi aratılınca çıkan kullanıcılar

11. 2 adet trigger tanımlamalı ve arayüzden girilecek değerlerle tetiklemelisiniz. Trigger'ın çalıştığına dair arayüze bilgilendirme mesajı döndürülmelidir. (arayüzden mesaj vermiyor onu vermesi sağlanacak)

```
CREATE OR REPLACE FUNCTION delete_species_if_no_breeds()
RETURNS TRIGGER AS $$
BEGIN
    -- Kontrol et: Bu türün başka breed'leri var mı?
    IF NOT EXISTS (SELECT 1 FROM breeds WHERE species_name = OLD.species_name) THEN
        -- Eğer başka breed yoksa, species'i sil
        DELETE FROM species WHERE species_name = OLD.species_name;
    END IF;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

-- Trigger'ı oluştur
CREATE TRIGGER delete_species_if_no_breeds
AFTER DELETE ON breeds
FOR EACH ROW
EXECUTE FUNCTION delete_species_if_no_breeds();
```

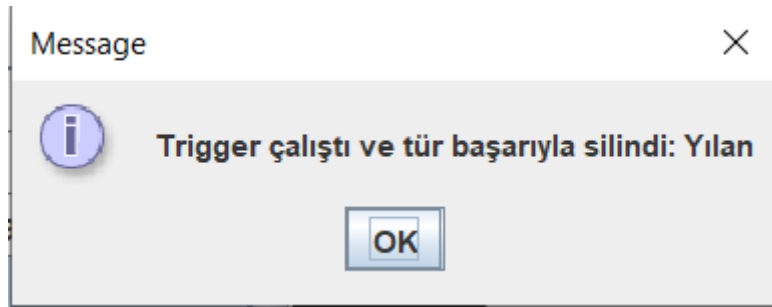
```
private void deleteBreed(String speciesName, String breedName) {
    String deleteBreedQuery = "DELETE FROM breeds WHERE breed_name = ?";
    breedName = formatInput(breedName);

    try (PreparedStatement stmt = Database.getConnection().prepareStatement(deleteBreedQuery)) {
        stmt.setString(1, breedName);
        int rowsAffected = stmt.executeUpdate();

        if (rowsAffected > 0) {
            if (isSpeciesDeleted(speciesName)) {
                JOptionPane.showMessageDialog(null, "Trigger çalıştı ve tür başarıyla silindi: " + speciesName);
            }
        } else {
            JOptionPane.showMessageDialog(null, "Seçilen breed silinemedi: " + breedName, "Hata", JOptionPane.ERROR_MESSAGE);
        }
    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(null, "Hata: Breed silinirken bir sorun oluştu.", "Hata", JOptionPane.ERROR_MESSAGE);
    }
}

private boolean isSpeciesDeleted(String speciesName) throws SQLException {
    String checkSpeciesQuery = "SELECT 1 FROM species WHERE species_name = ?";

    try (PreparedStatement stmt = Database.getConnection().prepareStatement(checkSpeciesQuery)) {
        stmt.setString(1, speciesName);
        try (ResultSet rs = stmt.executeQuery()) {
            return !rs.next();
        }
    }
}
```

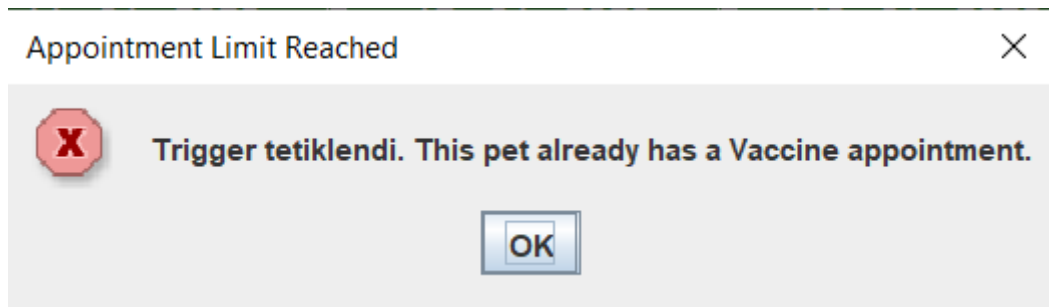



```
CREATE OR REPLACE FUNCTION check_appointment_limit() --Yeni bir randevu eklenmeden önce, aynı hayvan için aynı türde ve aynı tarihte bir randevunun zaten mevcut olup olmadığını
--kontrol eder.
RETURNS TRIGGER AS $$
BEGIN
    -- Aynı hayvan için aynı tür ve aynı tarihte bir randevu var mı kontrol et
    IF EXISTS (
        SELECT 1
        FROM appointments
        WHERE pet_id = NEW.pet_id
        AND appointment_date = NEW.appointment_date
        AND appointment_type = NEW.appointment_type
    ) THEN
        -- Hata mesajı fırlat ve SQLSTATE belirt
        RAISE EXCEPTION 'Trigger tetiklendi. This pet already has an appointment.'
        USING ERRCODE = '45000';
    END IF;

    -- İşleme devam et
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP FUNCTION IF EXISTS check_appointment_limit(); --check_appointment_limit adlı fonksiyonu veritabanından siler.

-- Trigger tanımla
CREATE TRIGGER check_appointment_limit_trigger --appointments tablosuna yeni bir randevu eklenmeden önce, check_appointment_limit fonksiyonunu çalıştırır.
BEFORE INSERT ON appointments
FOR EACH ROW
EXECUTE FUNCTION check_appointment_limit();
```



```
CREATE OR REPLACE FUNCTION create_vaccine_card() -- Bu işlem, her yeni evcil hayvan için bir aşı kartının başlangıçta otomatik olarak oluşturulmasını sağlar.
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO vaccine (pet_id, vaccine_date)
    VALUES (NEW.pet_id, NULL);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_pet_insert -- pets tablosuna her yeni bir kayıt eklendiğinde otomatik olarak create_vaccine_card fonksiyonunu çalıştırır
AFTER INSERT ON pets
FOR EACH ROW
EXECUTE FUNCTION create_vaccine_card();
```

```
if (generatedKeys.next()) {
    int petId = generatedKeys.getInt(1);

    if (isVaccineCardCreated(petId)) {
        JOptionPane.showMessageDialog(null, "The trigger worked and the vaccination card was created.");
    } else {
        JOptionPane.showMessageDialog(null, "Hayvan eklendi ancak aşı kartı oluşturulamadı.");
    }
}
```

Message



The trigger worked and the vaccination card was created.

OK